

MovieLens Movie Recommendation System

HarvardX Data Science Professional Certificate - Capstone Project (PH125.9x)

Tymofii Rzhevskyi

2026-09-15

Contents

1	Introduction	1
2	Data Preparation Process	2
3	Exploratory Data Analysis (EDA)	3
3.1	Basic Structure & Statistic Summary	3
3.2	Rating Distribution	5
3.3	Sparsity Analysis	7
3.4	Genre Distribution	9
3.5	Temporal Trends	11
3.6	Concentration of Rating Activity Across Users and Movies	13
3.7	Key Observations	17
4	Model Development	17
4.1	RMSE Definition	18
4.2	Baseline Model	18
4.3	Regularization of the Baseline Model	21
4.4	Matrix Factorization Model	23
5	Holdout Set Evaluation	26
6	Conclusion	27

1 Introduction

The primary objective of this project is to develop a movie recommendation system utilizing the well-established MovieLens dataset, collected and maintained by the GroupLens Research Laboratory at the University of Minnesota. The dataset comprises millions of ratings, scored by users to thousands of movies.

The recommendation systems have become an essential component of modern digital platforms, enabling users to discover relevant content across services such as Netflix and Amazon. To predict the content a user may find attractive the systems, at their cores, rely on patterns discernible in a way in which the user's data such as ratings, watch history, and number of replays were used.

The performance of the recommendation system model will be evaluated using Root Mean Squared Error (RMSE), which quantifies the average magnitude between the ratings predicted by the model itself and the actual ratings provided by users. A lower RMSE indicates that the model's predictions aligned accurately with the actual user ratings. The final recommendation system will be selected by comparing RMSE values across all tested models.

2 Data Preparation Process

The data preparation process begins with downloading the MovieLens dataset directly from the official GroupLens website. The following code was provided by the university. It undertakes the complete acquisition and initial partitioning of the original dataset, processes data to the structured format, and divides it into two independent subsets.

1. **edx** - used for model training and testing
2. **final_holdout_test** - reserved for the final evaluation of the best-performing model.

```
library(tidyverse)
library(skimr)
library(caret)
library(knitr)
library(recosystem)

#####
# Create edx and final_holdout_test sets
#####

# Note: this process could take a couple of minutes

if(!require(tidyverse)) install.packages("tidyverse", repos = "http://cran.us.r-project.org")
if(!require(caret)) install.packages("caret", repos = "http://cran.us.r-project.org")

# MovieLens 10M dataset:
# https://grouplens.org/datasets/movielens/10m/
# http://files.grouplens.org/datasets/movielens/ml-10m.zip

options(timeout = 120)

dl <- "ml-10M100K.zip"
if(!file.exists(dl))
  download.file("https://files.grouplens.org/datasets/movielens/ml-10m.zip", dl)

ratings_file <- "ml-10M100K/ratings.dat"
if(!file.exists(ratings_file))
  unzip(dl, ratings_file)

movies_file <- "ml-10M100K/movies.dat"
if(!file.exists(movies_file))
  unzip(dl, movies_file)

ratings <- as.data.frame(str_split(read_lines(ratings_file), fixed("::"), simplify = TRUE),
                        stringsAsFactors = FALSE)
colnames(ratings) <- c("userId", "movieId", "rating", "timestamp")
ratings <- ratings %>%
  mutate(userId = as.integer(userId),
         movieId = as.integer(movieId),
         rating = as.numeric(rating),
         timestamp = as.integer(timestamp))

movies <- as.data.frame(str_split(read_lines(movies_file), fixed("::"), simplify = TRUE),
                        stringsAsFactors = FALSE)
colnames(movies) <- c("movieId", "title", "genres")
```

```

movies <- movies %>%
  mutate(movieId = as.integer(movieId))

movielens <- left_join(ratings, movies, by = "movieId")

set.seed(1, sample.kind="Rounding")
test_index <- createDataPartition(y = movielens$rating, times = 1, p = 0.1, list = FALSE)
edx <- movielens[-test_index,]
temp <- movielens[test_index,]

final_holdout_test <- temp %>%
  semi_join(edx, by = "movieId") %>%
  semi_join(edx, by = "userId")

removed <- anti_join(temp, final_holdout_test)
edx <- rbind(edx, removed)

rm(dl, ratings, movies, test_index, temp, movielens, removed)

```

Prepared **edx** dataset is partitioned into a training set and a test set. Partition ensures that the model performance is evaluated against held-out data throughout the development phase, where the **final_holdout_test** set was not used. A conventional 80/20 split ratio is applied, allocating 80% of the **edx** observations to the training set and the remaining 20% for evaluation purposes.

```

set.seed(1, sample.kind = "Rounding")

index <- createDataPartition(edx$rating, times = 1, p = 0.2, list = FALSE)
edx_train_set <- edx[-index,]
edx_test_set <- edx[index,]

edx_train_set <- edx_train_set %>%
  semi_join(edx_test_set, by = "movieId")

edx_test_set <- edx_test_set %>%
  semi_join(edx_train_set, by = "movieId")

```

3 Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) is essential to perform before building a predictive model as it provides a thorough understanding of the dataset's structure and statistical distributions.

All analyses presented in this section are performed exclusively on the **edx** dataset, which represents 90% of the total available data. The **final_holdout_test** set is treated as completely unseen data and is withheld entirely from all stages of exploratory analysis.

3.1 Basic Structure & Statistic Summary

The exploration begins with examination of the **edx** dataset's basic structure. The first 10 observations are previewed using the **head()** function providing an initial overview of the dataset's structure. To obtain a more comprehensive structural overview, the **glimpse()** function from the **dplyr** library is utilized, exposing the dataset's dimensions alongside each column's name and associated data type.

The **skim()** function from the **skimr** library is applied for thorough statistical characterization. This function generates a detailed summary report encompassing column types, value frequencies, and descriptive statistics, where outputs organized by types.

```
head(edx, n = 9)
```

```

  userId movieId rating timestamp title
1      1     122      5 838985046 Boomerang (1992)
2      1     185      5 838983525 Net, The (1995)
4      1     292      5 838983421 Outbreak (1995)
5      1     316      5 838983392 Stargate (1994)
6      1     329      5 838983392 Star Trek: Generations (1994)
7      1     355      5 838984474 Flintstones, The (1994)
8      1     356      5 838983653 Forrest Gump (1994)
9      1     362      5 838984885 Jungle Book, The (1994)
10     1     364      5 838983707 Lion King, The (1994)

      genres
1      Comedy|Romance
2      Action|Crime|Thriller
4      Action|Drama|Sci-Fi|Thriller
5      Action|Adventure|Sci-Fi
6      Action|Adventure|Drama|Sci-Fi
7      Children|Comedy|Fantasy
8      Comedy|Drama|Romance|War
9      Adventure|Children|Romance
10 Adventure|Animation|Children|Drama|Musical

```

```
glimpse(edx)
```

Rows: 9,000,055

Columns: 6

```

$ userId   <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, ~
$ movieId  <int> 122, 185, 292, 316, 329, 355, 356, 362, 364, 370, 377, 420, ~
$ rating   <dbl> 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, ~
$ timestamp <int> 838985046, 838983525, 838983421, 838983392, 838983392, 83898~
$ title    <chr> "Boomerang (1992)", "Net, The (1995)", "Outbreak (1995)", "S~
$ genres   <chr> "Comedy|Romance", "Action|Crime|Thriller", "Action|Drama|Sci~

```

```
skim(edx)
```

Table 1: Data summary

Name	edx
Number of rows	9000055
Number of columns	6
Column type frequency:	
character	2
numeric	4
Group variables	None

Variable type: character

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
title	0	1	8	160	0	10676	0
genres	0	1	3	60	0	797	0

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
userId	0	1	3.586982e+00	1.585.25	1.0	18124	35738	53607	71567	
movieId	0	1	4.121700e+00	8942.11	1.0	648	1834	3626	65133	
rating	0	1	3.510000e+00	1.06	0.5	3	4	4	5	
timestamp	0	1	1.032616e+09	5966785.28	965200994	16768283	1035493918	1267508812	131131736	

The **edx** dataset consists of 9,000,055 observations across 6 columns, where each row represents discrete rating event assigned to movie by user.

The dataset contain 2 character columns and 4 numerical. Inspection of a **character** columns reveals that the **title** column holds 10,676 unique values, with title length ranging from 8 up to 160 characters. The **genres** column contains 797 unique values, with string lengths ranging from 3 to 60 characters.

Inspection of a numeric columns reveals that 3 columns are of an integer type and one is of a double type. All numeric variables exhibit a completeness rate of 1.0, confirming the absence of missing values. Five distributional percentiles are reported for each numeric column, where **p0** and **p100** correspond to the minimum and maximum observed values, respectively.

These structural characteristics confirm that scale of dataset is sufficient to support the identification of statistically meaningful patterns. The observed range in rating values further suggests that both user and movie effects are likely present and must be accounted for in future model development.

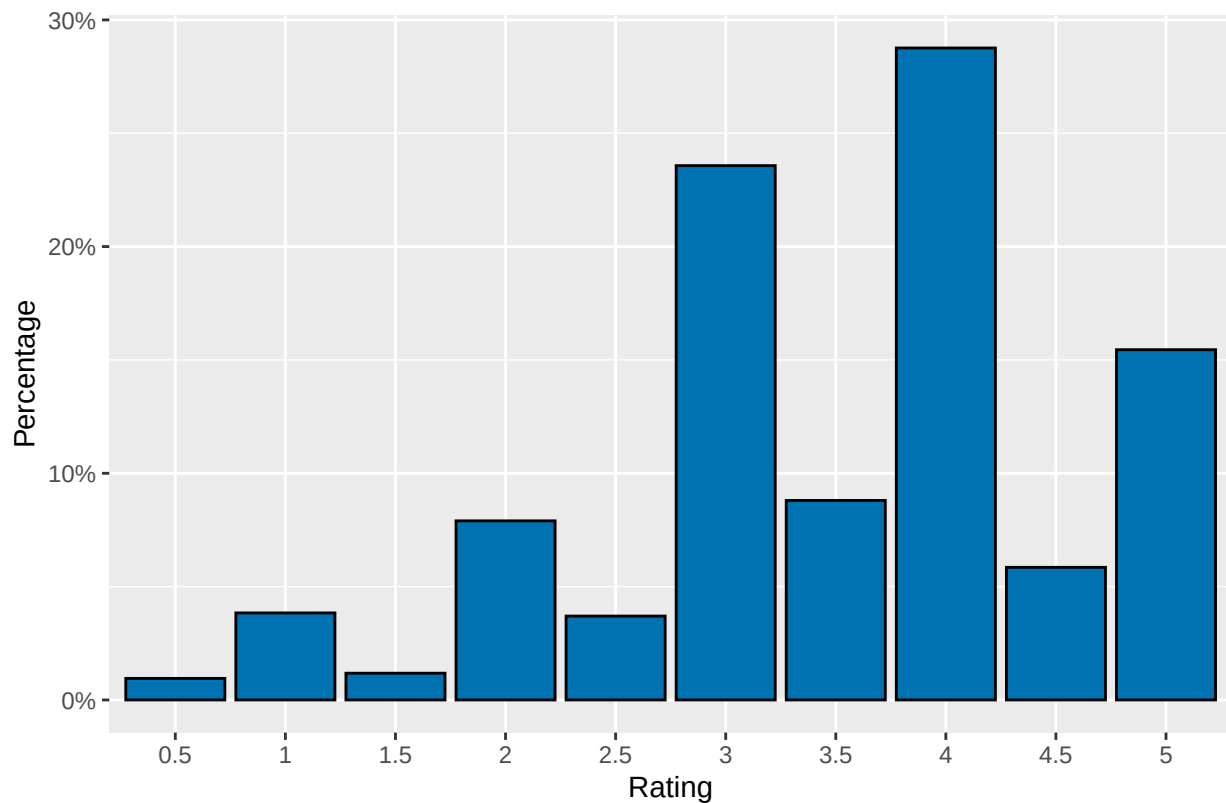
3.2 Rating Distribution

The distribution of ratings across dataset provides a foundational step in understanding user rating behavior. Examine the frequency of assigned ratings enables the identification of systematical biases and structural tendencies that predictive model must account for. The histogram below illustrates the frequency of each discrete rating values ranging from from 0.5 to 5.0.

```
edx_rating <- edx %>%
  count(rating, name = "num_rating") %>%
  mutate(percentage = round(num_rating / sum(num_rating) * 100, 2)) %>%
  arrange(desc(rating))

edx_rating %>%
  ggplot(aes(x = factor(rating), y = percentage)) +
  geom_col(fill = "#0072B2",
           color = "black") +
  scale_y_continuous(labels = function(x) paste0(x, "%")) +
  labs(
    title = "Distribution of Ratings",
    x = "Rating",
    y = "Percentage"
  )
```

Distribution of Ratings



```
edx_rating %>%
  mutate(percentage = sprintf("%.2f%%", percentage))
```

	rating	num_rating	percentage
1	5.0	1390114	15.45%
2	4.5	526736	5.85%
3	4.0	2588430	28.76%
4	3.5	791624	8.80%
5	3.0	2121240	23.57%
6	2.5	333010	3.70%
7	2.0	711422	7.90%
8	1.5	106426	1.18%
9	1.0	345679	3.84%
10	0.5	85374	0.95%

```
edx_rating %>%
  summarise(Above_three = sprintf("%.2f%%", round(sum(percentage[1:5]))),
            Below_three = sprintf("%.2f%%", round(sum(percentage[6:10]))))
```

	Above_three	Below_three
1	82.00%	18.00%

The distribution reveals a clear preference for whole numbers with values such as 3.0, 4.0, and 5.0 which are occurring more frequently than ratings such as 0.5, 1.5, or 2.5. The overall distribution is left-skewed with the majority of ratings centered in the 3.0 to 4.0 range. This pattern suggests that users tend to assign favorable ratings more frequently than unfavorable ones.

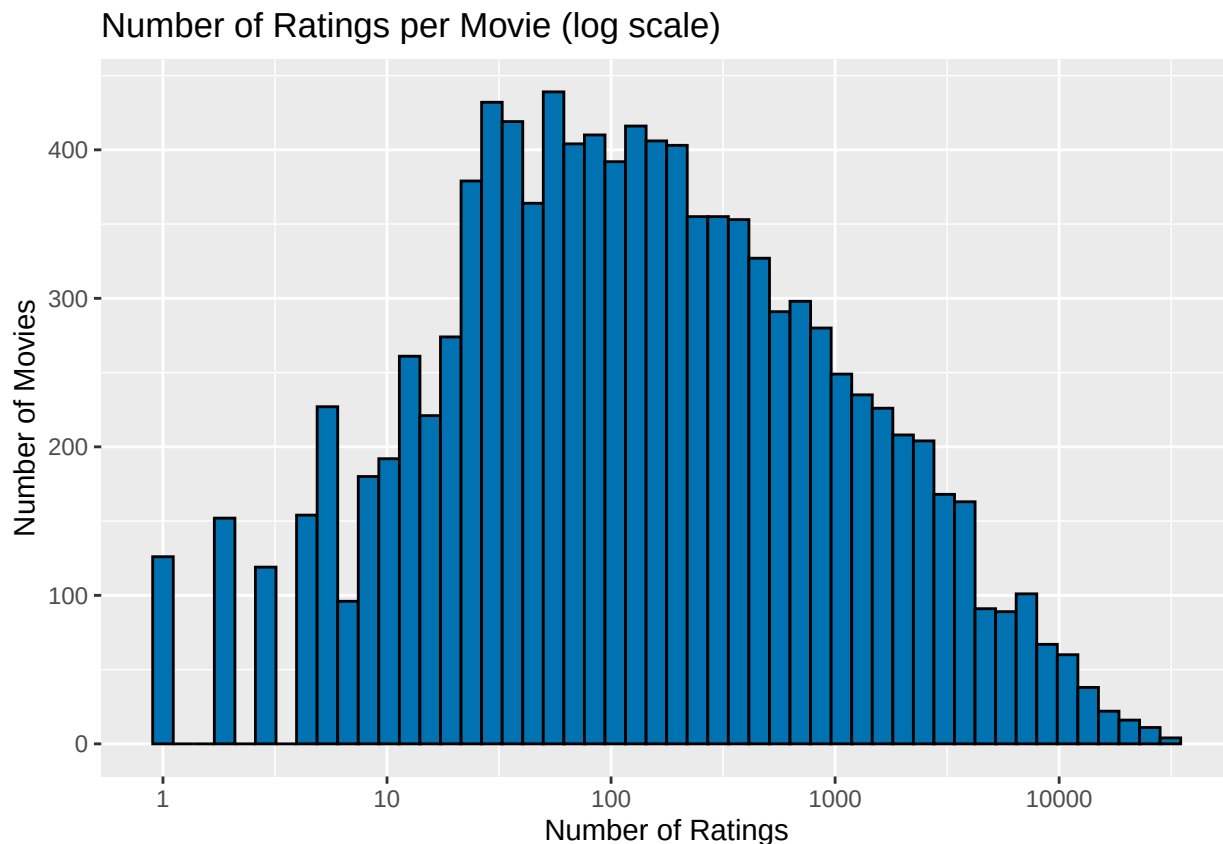
3.3 Sparsity Analysis

The core of every recommendation system lies in a fundamental challenge known as data sparsity. The user-movie rating matrix is a theoretical structure in which each row corresponds to a unique user and each column to distinct movie. Individual cells represent assigned ratings but most are empty as no user is expected to have rated every available movie. That emptiness is referred to as *sparsity*.

3.3.1 Ratings per Movie

The analysis quantifies the number of ratings received per movie and visualises the resulting distribution as a histogram. Given the **left-skewed** distribution, a logarithmic scale is applied to the x-axis to facilitate a more accurate representation.

```
edx |>
  count(movieId, name = "n_ratings") |>
  ggplot(aes(n_ratings)) +
  geom_histogram(bins = 50,
                 fill = "#0072B2",
                 color = "black") +
  scale_x_log10() +
  labs(
    title = "Number of Ratings per Movie (log scale)",
    x = "Number of Ratings",
    y = "Number of Movies"
  )
```



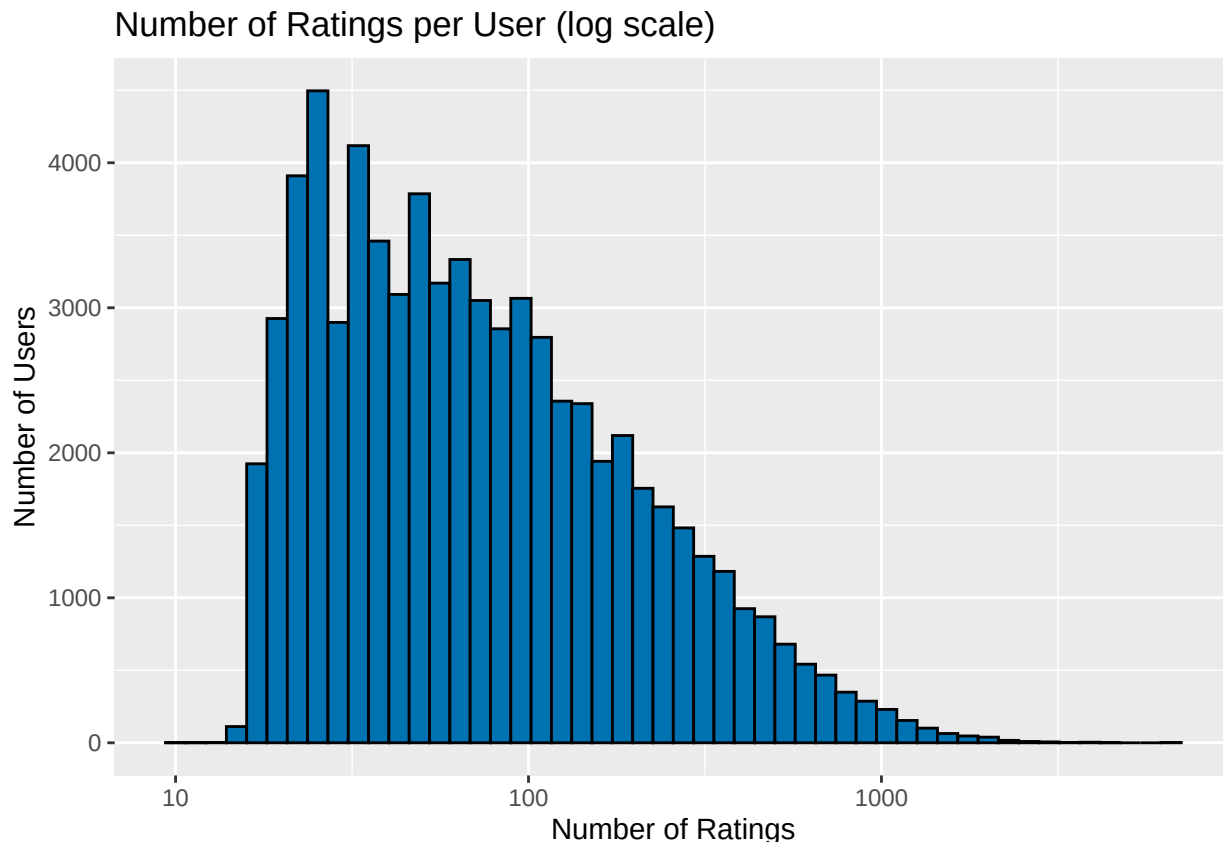
The resulting distribution confirms that movie popularity varies across the dataset. A subset of highly popular titles accumulates thousands of ratings, while a substantial proportion of less prominent movies received only a handful of observations. This indicates an imbalance in the dataset and motivates inclusion

of a movie bias term in future models.

3.3.2 Ratings per User

An analogous analysis is applied to users with a notable distinction in distributional behavior. Although a logarithmic scale is applied on the x-axis, but the distribution remains **right-skewed**. This reflects an extreme imbalance in user activity. The dataset is dominated by users who have rated a relatively small number of movies, while only a minor fraction of users have contributed to 500 movies or more. The inequality in engagement is sufficiently expressed that the logarithmic compression fails to fully normalize the distribution. These findings motivates the use of regularization in later models.

```
edx |>
  count(userId, name = "n_ratings") |>
  ggplot(aes(n_ratings)) +
  geom_histogram(bins = 50,
                 fill = "#0072B2",
                 color = "black") +
  scale_x_log10() +
  labs(
    title = "Number of Ratings per User (log scale)",
    x = "Number of Ratings",
    y = "Number of Users"
  )
```



The distribution of ratings per user reveals a wide range. Users have submitted as few as approximately 10 ratings while highly engaged individuals have contributed over 1,000. The distribution falls within a 20 to 30 rating range indicating this interval as the most prevalent level of user engagement. The distribution remains **right-skewed** even after the logarithmic transformation reflecting that the majority of users have

contributed relatively few ratings while only a small minority have remained highly active.

3.3.3 Sparsity of the User-Movie Matrix

After examining both distributions individually, quantifying the overall sparsity of the user-movie matrix becomes possible. Sparsity is defined as the proportion of all theoretical user-movie combinations for which no rating has been observed.

```
num_users <- n_distinct(edx$userId)
num_movies <- n_distinct(edx$movieId)
num_ratings <- nrow(edx)

sparsity <- 1 - (num_ratings / (num_users * num_movies))

tibble(
  "Unique Users" = num_users,
  "Unique Movies" = num_movies,
  "Unique Ratings" = num_ratings,
  "Possible Combinations" = num_users * num_movies,
  "Sparsity" = paste0(round(sparsity * 100, 2), "%")
) %>%
  knitr::kable(caption = "Sparsity of the User-Movie Rating Matrix")
```

Table 4: Sparsity of the User-Movie Rating Matrix

Unique Users	Unique Movies	Unique Ratings	Possible Combinations	Sparsity
69878	10677	9000055	746087406	98.79%

The computed sparsity of the user-movie rating matrix is 98.79%, indicating that only 1.21% of all possible user-movie interactions are observed within the dataset. This confirms the matrix’s extreme sparsity a characteristic that presents significant challenges for collaborative filtering approaches and emphasises the importance of robust regularization and bias importance in the modeling pipeline.

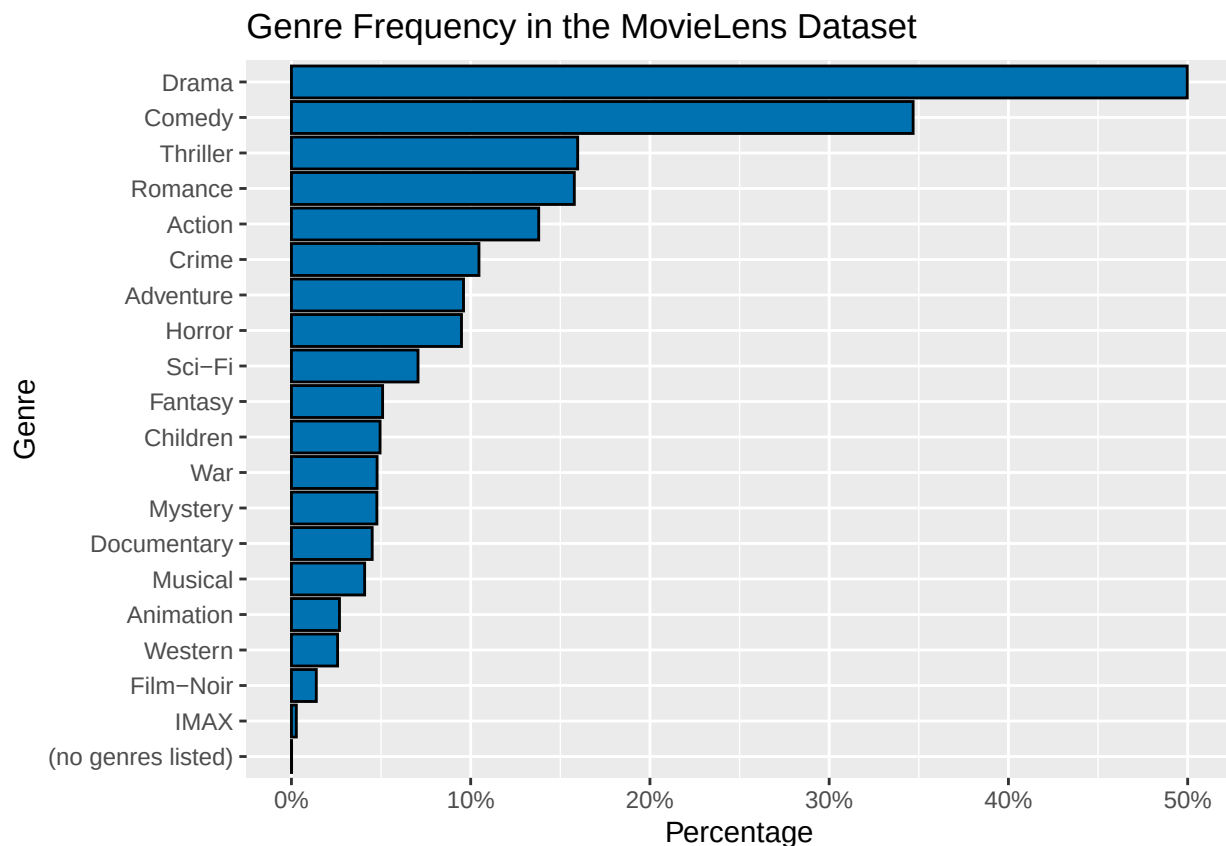
3.4 Genre Distribution

Within the MovieLens dataset, each movie is assigned one or more genre labels stored as a single pipe-delimited string rather than as discrete categorical variables. For example, a movie classified under multiple genres may have a combined label such as **“Children|Comedy|Fantasy”**. Applying frequency analysis directly to these strings would treat each unique combination as a single distinct category, leading to a substantially underestimation of the true prevalence of individual genres across the dataset. To ensure accurate genre level analysis, each composite genre string must therefore be disaggregated into its constituent labels prior to computing genre frequencies, allowing every genre to be counted independently regardless of the combination in which it appears.

```
n_movies <- edx %>%
  distinct(movieId) %>%
  nrow()

edx_genre <- edx %>%
  distinct(movieId, genres) %>%
  separate_rows(genres, sep = "\\|") %>%
  count(genres, name = "num_movies") %>%
  mutate(percentage = num_movies / n_movies * 100) %>%
  arrange(desc(percentage))
```

```
edx_genre %>%
  ggplot(aes(x = percentage, y = reorder(genres, percentage))) +
  geom_col(stat = "identity", fill = "#0072B2", color = "black") +
  scale_x_continuous(labels = function(x) paste0(x, "%")) +
  labs(
    title = "Genre Frequency in the MovieLens Dataset",
    x = "Percentage",
    y = "Genre"
  )
)
```



```
edx_genre %>%
  mutate(percentage = sprintf("%.2f%%", percentage))
```

```
# A tibble: 20 x 3
  genres          num_movies percentage
  <chr>          <int> <chr>
1 Drama             5336 49.98%
2 Comedy            3703 34.68%
3 Thriller           1705 15.97%
4 Romance           1685 15.78%
5 Action            1473 13.80%
6 Crime             1117 10.46%
7 Adventure          1025  9.60%
8 Horror             1013  9.49%
9 Sci-Fi              754  7.06%
10 Fantasy            543  5.09%
11 Children           528  4.95%
```

12 War	510 4.78%
13 Mystery	509 4.77%
14 Documentary	481 4.51%
15 Musical	436 4.08%
16 Animation	286 2.68%
17 Western	275 2.58%
18 Film-Noir	148 1.39%
19 IMAX	29 0.27%
20 (no genres listed)	1 0.01%

The resulting distribution reveals a clear enumeration of genre prevalence within the dataset. **Drama** and **Comedy** emerge as the most frequently occurring genres by a considerable margin, appearing in 49.9% and 34.6% of item records respectively. Specialized genres such as **Documentary** and **Film-Noir** represent the rarest categories, each constituting less than 5% of the collection, suggesting that genre level differences may contribute to systematic variations in rating patterns and motivates the potential inclusion of a genre specific bias term in more advanced modeling frameworks.

3.5 Temporal Trends

The `timestamp` column records the exact moment at which each rating was submitted in a Unix timestamp format representing the number of seconds elapsed since January 1, 1970 (UTC). Converting this column to a human readable datetime format enables temporal analysis of rating behavior and facilitates the identification of time dependent patterns in user activity.

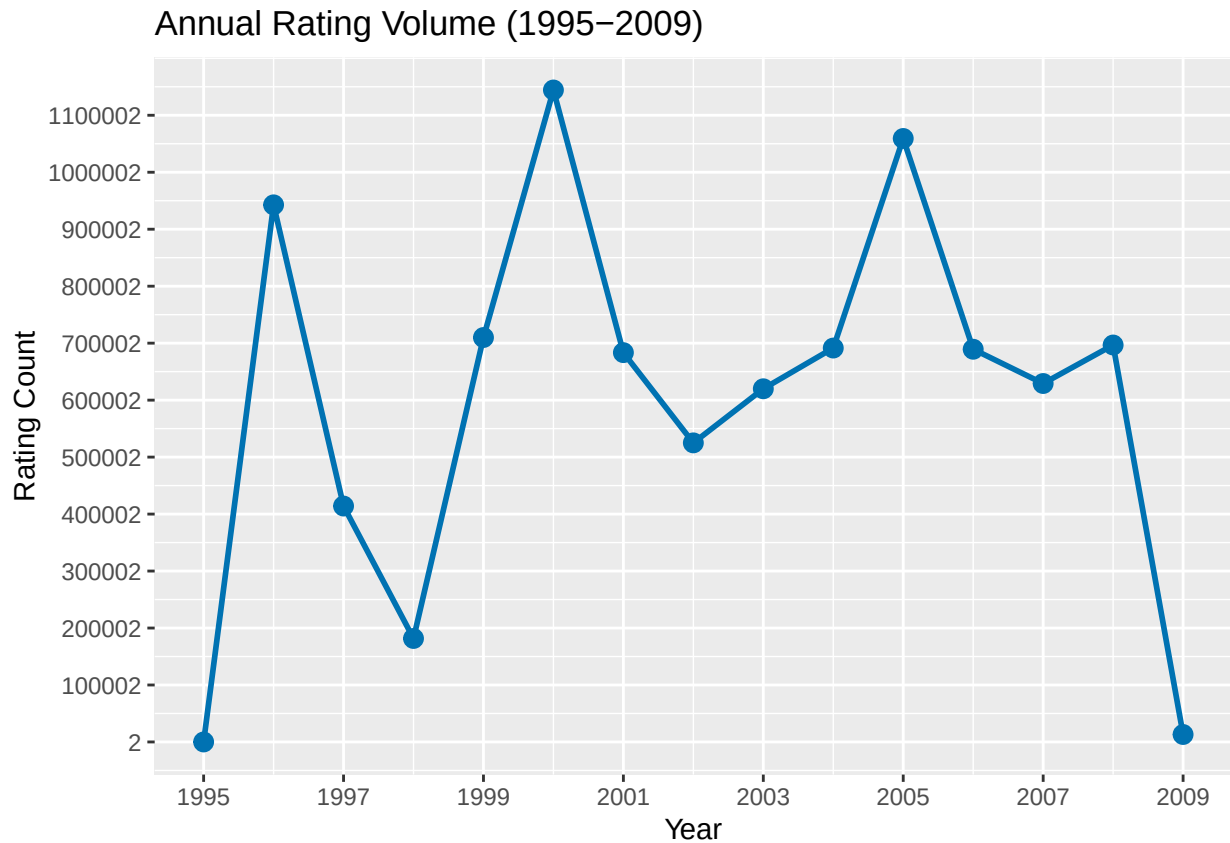
This transformation is significant as rating activity cannot be assumed to be stationary over time. User preferences, evaluation standards, and levels of engagement may shift across different periods. Consequently, subsequent predictive models may need to account for temporal bias or time-dependent effects to achieve accurate performance.

```
edx_temp <- edx %>%
  mutate(date = as_datetime(timestamp),
         year = year(date))

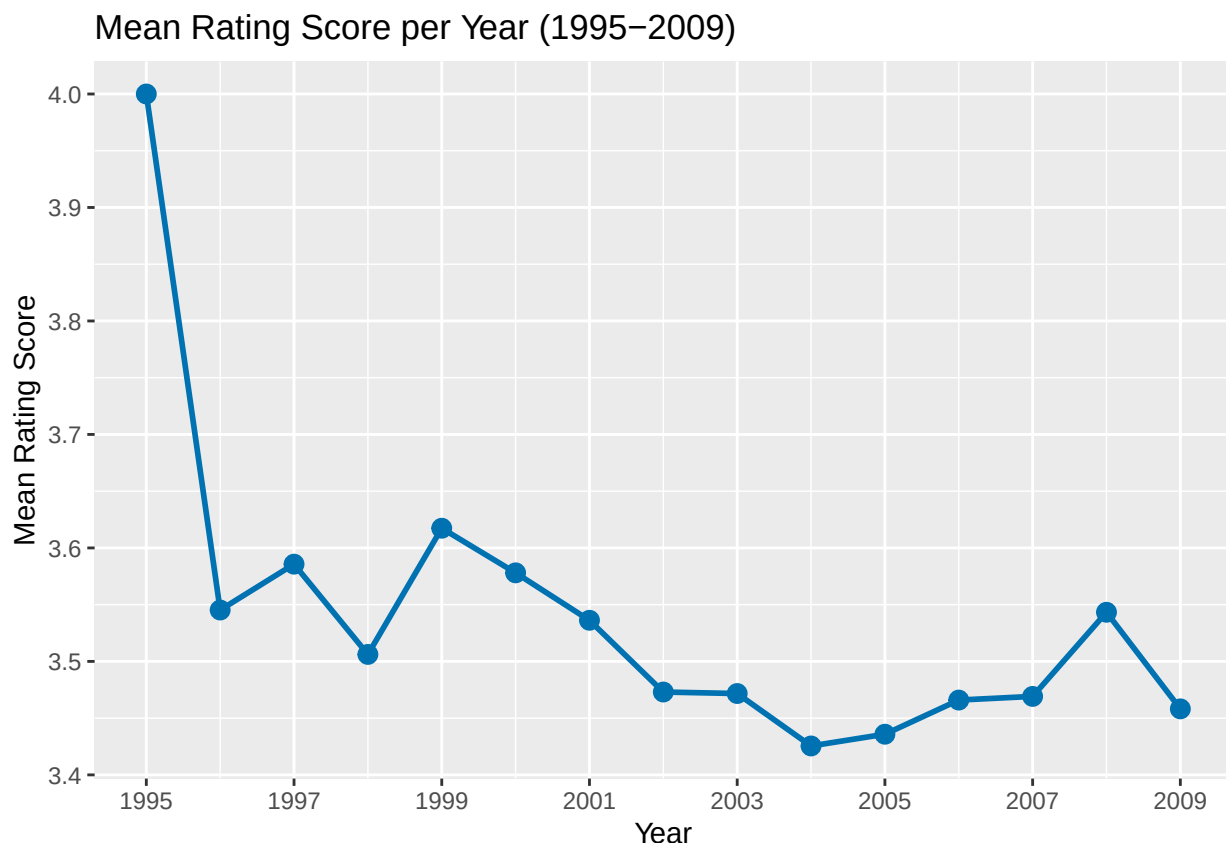
ratings_per_year <- edx_temp %>%
  group_by(year) %>%
  summarise(num_ratings = n(),
            mean_rating = mean(rating))
```

Following the datetime conversion, the next step is to plot **annual rating volume** and **mean rating score per year**:

```
ratings_per_year %>%
  ggplot(aes(x = year, y = num_ratings)) +
  geom_point(color = "#0072B2", size = 3) +
  geom_line(color = "#0072B2", linewidth = 1) +
  scale_x_continuous(breaks = round(seq(min(ratings_per_year$year),
                                         max(ratings_per_year$year), by = 2), 1)) +
  scale_y_continuous(breaks = round(seq(min(ratings_per_year$num_ratings),
                                         max(ratings_per_year$num_ratings), by = 100000), 1)) +
  labs(
    title = "Annual Rating Volume (1995-2009)",
    x = "Year",
    y = "Rating Count"
  )
```



```
ratings_per_year %>%
  ggplot(aes(x = year, y = mean_rating)) +
  geom_point(color = "#0072B2", size = 3) +
  geom_line(color = "#0072B2", linewidth = 1) +
  scale_x_continuous(breaks = round(seq(min(ratings_per_year$year),
                                         max(ratings_per_year$year), by = 2), 1)) +
  scale_y_continuous(breaks = round(seq(min(ratings_per_year$mean_rating),
                                         max(ratings_per_year$mean_rating), by = 0.09), 1)) +
  labs(
    title = "Mean Rating Score per Year (1995-2009)",
    x = "Year",
    y = "Mean Rating Score"
  )
)
```



The first plot reveals that rating activity was not uniformly distributed over time. Certain years have substantially higher ratings volumes, attributable to the growing popularity of a rating platform during specific periods or the releases of major titles that attracted widespread user engagement.

The second plot examines whether the mean rating score has evolved over time. An upward or downward trend in mean rating score would indicate a temporal bias in the data suggesting that the period in which a rating was submitted carries additional exploratory intensity beyond a characteristics of the users or movies. The resulting time series plot reveals four notable observations. The first is an overall downward trend in the mean rating scores across the full temporal span of the dataset. The second is near perfect stability in mean rating scores during two sub-periods 2002-2003 and 2006-2007. The third is an abrupt upward shift from 2007 to 2008. The fourth reflects a sharp decline from 2008 to 2009.

Examination of the `timestamp` column provides several important insights into the temporal dynamics of the dataset. Notably, rating activity is not stationary over time. Major cyclical peaks in annual rating volume are observed, reaching approximately 1.1 million ratings in 2000 and exceeding 1 million in 2005. In contrast, the mean rating score followed a sustained downward trend over the same time period, declining from approximately 4.0 in 1995 to roughly 3.45 by 2009. Additionally, high irregularity is observed in the rating activity, particularly in the near-zero values at the series boundaries in 1995 and 2009.

3.6 Concentration of Rating Activity Across Users and Movies

A fundamental characteristic of recommendation datasets is pronounced unevenness in rating activity. A disproportionately small subset of users and movies collectively accounts for the overwhelming majority of observed interactions. This phenomenon is closely related to **power law distributions** and has direct implications for model design. Recommendation system must be capable of generating reliable predictions not only for the well represented, highly active users and popular titles, for the large and sparsely observed population of inactive users and obscure movies as well.

This section quantifies the concentration of rating activity from two complementary perspectives. The first

examines the distribution of the total volume of ratings across user percentiles. The second captures the proportion of ratings attributed to titles at varying points along the popularity distribution.

3.6.1 Preparation

Prior to the concentration analysis, summary tables of rating counts are constructed separately for users and movies, sorted in descending order of activity. Cumulative rating totals are appended to each table to facilitate the computation of proportional share across ranked segments of the population.

```
user_activity <- edx %>%
  group_by(userId) %>%
  summarize(rating_count = n(), .groups = "drop") %>%
  arrange(desc(rating_count)) %>%
  mutate(cumsum_ratings = cumsum(rating_count))

item_popularity <- edx %>%
  group_by(movieId) %>%
  summarize(rating_count = n(), .groups = "drop") %>%
  arrange(desc(rating_count)) %>%
  mutate(cumsum_ratings = cumsum(rating_count))

total_ratings <- sum(user_activity$rating_count)
```

3.6.2 User Activity Concentration

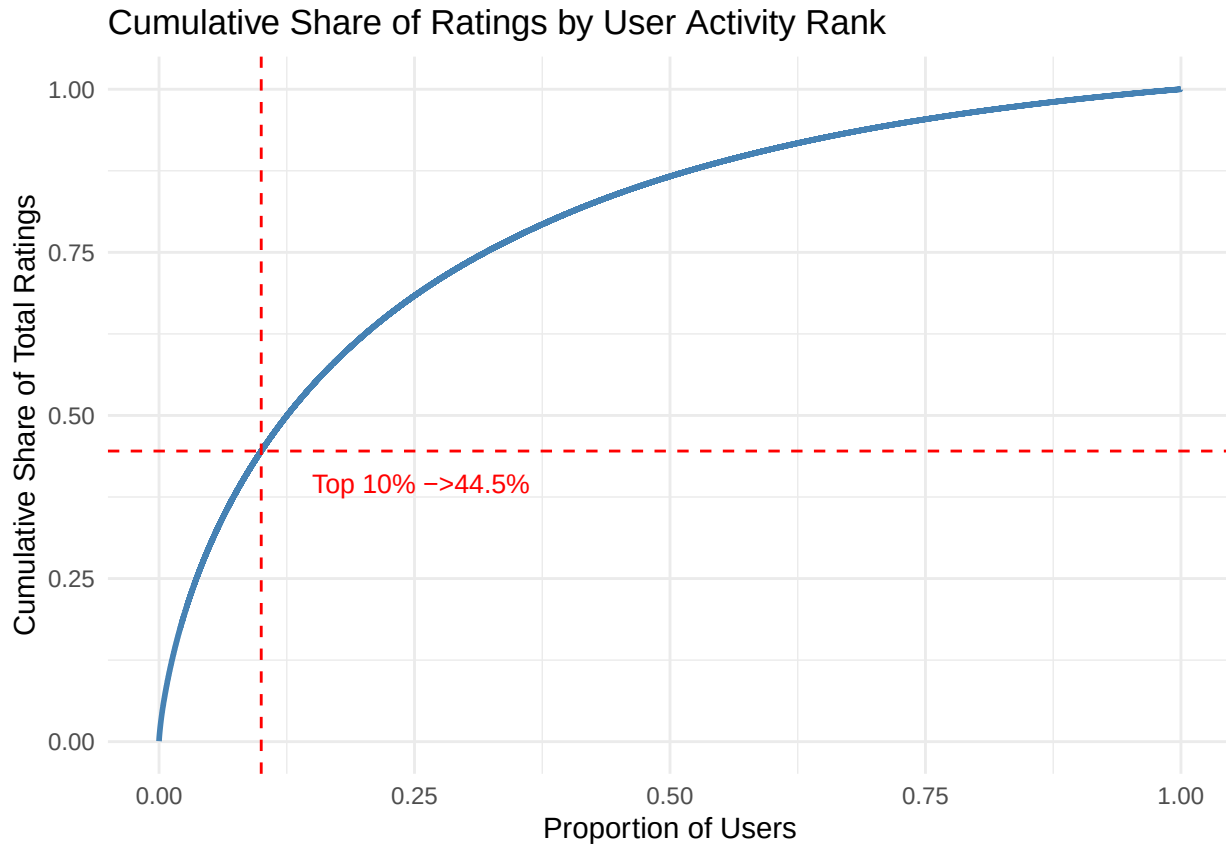
The concentration of rating activity among users is examined by quantifying the proportion of total ratings attributable to the most active segment of the user population and the maximum rating count observed within the least active segment.

```
top_10pct_users <- round(0.1 * nrow(user_activity))
top_10_share <- user_activity %>%
  slice(1:top_10pct_users) %>%
  summarize(share = sum(rating_count) / total_ratings) %>%
  pull(share)

bottom_30pct_users <- floor(0.3 * nrow(user_activity))
max_bottom_30_users <- user_activity %>%
  arrange(rating_count) %>%
  slice(1:bottom_30pct_users) %>%
  summarise(max_val = max(rating_count)) %>%
  pull(max_val)

user_activity %>%
  mutate(
    user_rank = row_number() / n(),
    cum_share = cumsum_ratings / total_ratings
  ) %>%
  ggplot(aes(x = user_rank, y = cum_share)) +
  geom_line(color = "steelblue", linewidth = 1) +
  geom_vline(xintercept = 0.1, linetype = "dashed", color = "red") +
  geom_hline(yintercept = top_10_share, linetype = "dashed", color = "red") +
  annotate("text", x = 0.15, y = top_10_share - 0.05,
    label = paste0("Top 10% ->",
      round(top_10_share * 100, 1), "%"),
    color = "red", size = 3.5, hjust = 0) +
```

```
labs(
  title = "Cumulative Share of Ratings by User Activity Rank",
  x     = "Proportion of Users",
  y     = "Cumulative Share of Total Ratings"
) +
theme_minimal()
```



The concentration analysis reveals a pronounced asymmetry in rating activity across the user population indicating a significant user activity imbalance. The top 10% of most active users account for approximately 44.54% of all observed ratings, suggesting that nearly half of all interactions originate from a minority of users. This skewed distribution may lead to systematically degraded and less personalized recommendation model. Conversely the bottom 30% of users with the lowest engagement levels, have rated at most 36 titles. This observation confirms that a substantial proportion of the user population has provided limited observational evidence, making it difficult to estimate reliable user specific preference parameters. This degree of concentration in user activity necessitates regularization in the modeling framework.

3.6.3 Item Popularity Concentration

A similar concentration analysis is applied to movies, quantifying the proportion of total ratings attributable to the most popular titles and characterizing the degree of rating scarcity observed at the lower tail of the movie popularity distribution.

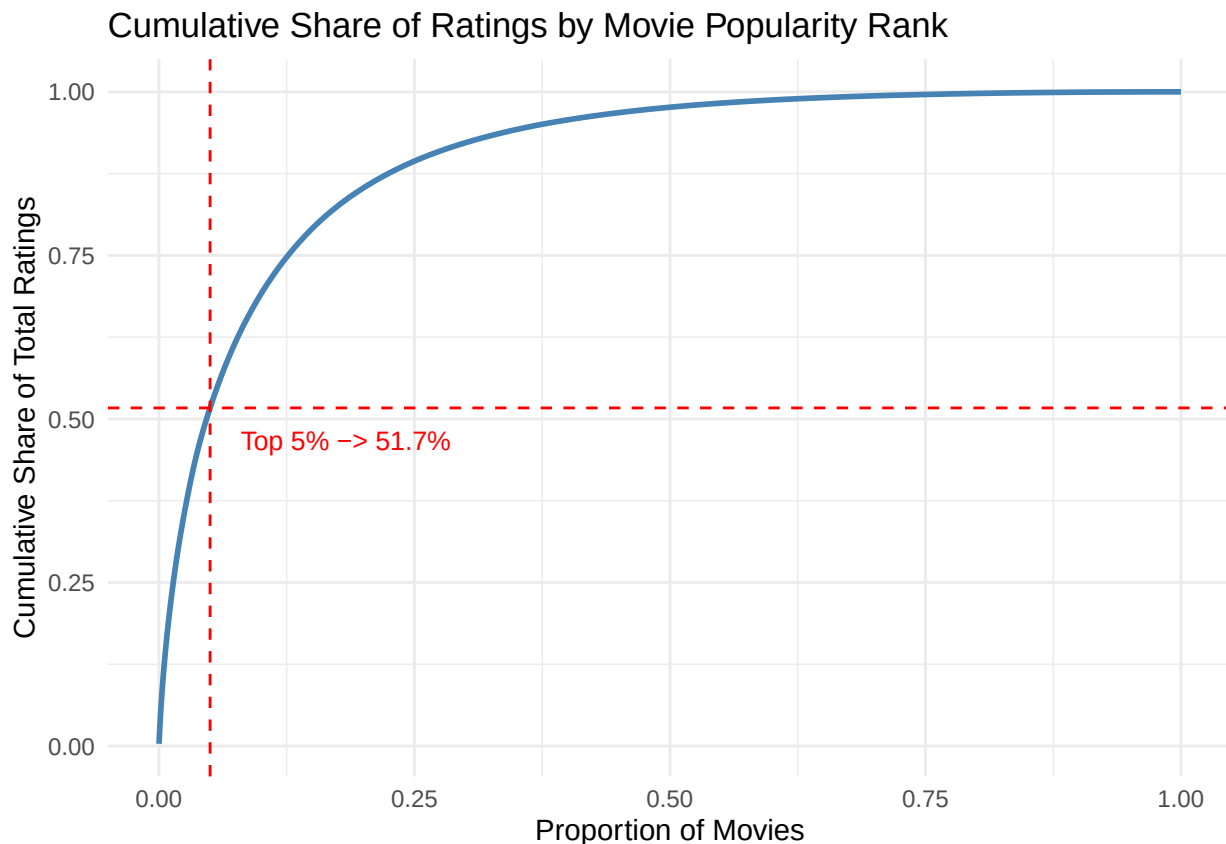
```
top_5pct_items <- round(0.05 * nrow(item_popularity))
top_5_share <- item_popularity %>%
  slice(1: top_5pct_items) %>%
  summarize(share = sum(rating_count) / total_ratings) %>%
  pull(share)
```

```

bottom_20pct_items <- floor(0.2 * nrow(item_popularity))
max_bottom_20_items <- item_popularity %>%
  arrange(rating_count) %>%
  slice(1:bottom_20pct_items) %>%
  summarise(max_val = max(rating_count)) %>%
  pull(max_val)

item_popularity %>%
  mutate(
    item_rank = row_number() / n(),
    cum_share = cumsum_ratings / total_ratings
  ) %>%
  ggplot(aes(x = item_rank, y = cum_share)) +
  geom_line(color = "steelblue", linewidth = 1) +
  geom_vline(xintercept = 0.05, linetype = "dashed", color = "red") +
  geom_hline(yintercept = top_5_share, linetype = "dashed", color = "red") +
  annotate("text", x = 0.08, y = top_5_share - 0.05,
    label = paste0("Top 5% → ",
      round(top_5_share * 100, 1), "%"),
    color = "red", size = 3.5, hjust = 0) +
  labs(
    title = "Cumulative Share of Ratings by Movie Popularity Rank",
    x = "Proportion of Movies",
    y = "Cumulative Share of Total Ratings"
  ) +
  theme_minimal()

```



The concentration of rating activity across movies is more pronounced than that observed among users. The top 5% of the most popular movies account for approximately 51.69% of all observed ratings, indicating that more than half of all interactions are attributable to just one in twenty movies in the dataset. Conversely, the bottom 20% of movies with fewer ratings and significantly less popularity, comprise 2,135 titles and obtain only 23 ratings, placing them in the sparse tail of the popularity distribution.

This concentration has two important implications for the modeling framework. First, rating estimates derived from the most popular title will be highly reliable as they are based on a large numbers of observations. Second, rating estimates for the least popular titles will be highly uncertain and this motivates the application of regularization to shrink these estimates towards global mean.

3.7 Key Observations

The exploratory data analysis (EDA) yields a number of structural characteristics that inform the approach to dataset modeling.

Firstly, rating behavior is systematically skewed toward higher values. Whole star ratings such as 3.0, 4.0 and 5.0 are selected more frequently than their half star counterparts indicating a clear user preference of round number ratings. This imbalance implies that model estimates will naturally be biased towards the dense region of rating space with a risk of underestimating the informativeness of low ratings and producing inaccurate predictions in that range.

Secondly, the user-movie matrix is extremely sparse with only 1.21% of all possible user-movie pairs observed. This level of sparsity precludes the use of memory-based methods, which require substantial rating overlap between users and instead motivates the adoption of model-based approaches capable of learning from limited observations.

Genre analysis reveals that Drama and Comedy dominate the dataset appearing at 49.9% and 34.6% of item records respectively, while specialized genres such as Documentary and Film-Noir collectively account for less than 5%. This distributional imbalance suggests that genre effect may constitute a meaningful predictive component. Temporal analysis further complicates the picture by revealing noise in the data. Mean rating scores exhibit an overall downward trend declining from 4.0 in 1995 to approximately 3.45 in 2009, while annual rating activity fluctuates considerably peaking at over 1.1 million ratings in 2000 and approaching near zero activity in 1995 and 2009 at either boundaries. These patterns suggest that temporal effect carries meaningful explanatory power beyond individual user and movie characteristics.

Concentration analysis confirms that both users and movies follow a long tail distribution. The top 10% of most active users account for 44.54% of all ratings, while the bottom 30% have rated only 36 titles. Analogously, the top 5% of movies account for 51.69% of all ratings, whereas the bottom 20% of movies receive approximately 23 ratings in total. These findings demonstrate that any model must remain robust to sparse observations from low activity users and low popularity titles, and not merely perform well for the best represented segments of the data.

Taken together, these findings indicate that naive approaches, such as simple global averaging or similarity-based methods, would perform poorly given the scale of sparsity and the extreme concentration of activity across both users and movies. This motivates a modeling framework that combines movie and user bias terms to capture rating tendencies, regularization to prevent overfitting to sparse observations and matrix factorization to improve prediction accuracy.

4 Model Development

The structural characteristics identified during exploratory data analysis delineate a clear set of constraints on model design. Rather than fitting a single undifferentiated model to the entire rating matrix, the modeling strategy implemented here decomposes the prediction problem into a sequence of additive components, each reflecting a distinct source of systematic variation identified during exploratory analysis. This section operationalizes those findings into a progression of models of increasing complexity, beginning with a global

mean estimator and extending at each step through the incremental incorporation of additional bias and regularization terms. All models will be developed and evaluated on an internal training and test sets derived from the `edx` dataset with the `final_holdout_test` set reserved for a single evaluation of the best performing model.

4.1 RMSE Definition

Formally, $Y_{u,i}$ denotes the observed rating assigned by user u to movie i , and $\hat{Y}_{u,i}$ denotes the corresponding model prediction. Predictive accuracy is quantified through the Root Mean Squared Error (RMSE), defined as:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{u,i} (\hat{Y}_{u,i} - Y_{u,i})^2}$$

where N denotes the total number of observed user-movie combinations, and the summation is taken over all such pairs.

```
RMSE <- function(true_ratings, predicted_ratings){
  sqrt(mean((predicted_ratings - true_ratings)^2))
}
rmse_result <- tibble(Model = character(), RMSE = numeric())
```

4.2 Baseline Model

The models developed in this section follow a progressive construction method advancing from the simplest possible estimator to a fully regularized, multiple effect model. Each intermediate model is evaluated on the test set prior to the introduction of the subsequent term. This approach ensures that every increase in model complexity is empirically justified and purposefully oriented toward achieving optimal predictive performance.

4.2.1 Global Mean model

The global mean model represents the simplest possible baseline model. It assigns an identical predicted ratings to every user-movie pair, regardless of the specific user or title involved. This type of model treats all observed deviations as difference as random variation. The model takes the following form:

$$Y_{u,i} = \mu + \epsilon_{u,i}$$

The error terms $\epsilon_{u,i}$ are assumed to be independent and drawn from a distribution centered at 0. μ denotes the true rating for all user-movie pairs. It is well established that the estimate minimizing the RMSE corresponds to the least squares estimate of μ , which is the average of all observed ratings:

```
mu_hat <- mean(edx_train_set$rating, na.rm = TRUE)
global_rmse <- RMSE(edx_test_set$rating, mu_hat)
rmse_result <- rmse_result %>% add_row(Model = "Global Mean:", RMSE = global_rmse)

cat(sprintf("Model RMSE: %.2f\n", global_rmse))
```

Model RMSE: 1.06

The global mean model predicts and assigns the constant prediction μ to every user-movie pair. Achieving an RMSE of 1.06 on the test set. Given that this model ignores all user or movie specific effects, its relatively high RMSE reflects the substantial variability in observed rating behavior that a single global mean is inherently incapable of capturing. This result establishes the baseline benchmark against which all subsequent model are assessed. Any model yielding an RMSE below 1.06 is considered to represent a meaningful improvement.

4.2.2 Movie Effect Model

The global mean model assumes that all movies share the same expected rating, represented by the overall mean μ of all observed ratings. However, certain titles systematically attract higher or lower ratings than the global mean, indicating that rating behavior is not uniform across movies. To account for this variation, the baseline model is extended by incorporating a movie bias term b_i , which captures the average deviation of a given movie's ratings from the overall mean μ . The model can therefore be written as:

$$Y_{u,i} = \mu + b_i + \epsilon_{u,i}$$

```
movie_effect <- edx_train_set %>%
  group_by(movieId) %>%
  summarise(b_i = mean(rating - mu_hat, na.rm = TRUE))

pred_movie <- edx_test_set %>%
  left_join(movie_effect, by = "movieId") %>%
  mutate(pred = mu_hat + b_i) %>%
  pull(pred)

movie_effect_rmse <- RMSE(edx_test_set$rating, pred_movie)
rmse_result <- rmse_result %>% add_row(Model = "Movie Effect Model:", RMSE = movie_effect_rmse)

cat(sprintf("Movie effect model RMSE: %.3f\n", movie_effect_rmse))
```

Movie effect model RMSE: 0.944

The movie effect model advances beyond the global mean baseline by capturing systematic deviations in ratings at the movie level. Rather than relying on a constant global estimate, predictions are formed by adjusting the overall mean according to each movie's characteristic deviation. Incorporating this bias correction reduces prediction error, confirming that the global mean baseline fails to account for movie specific bias in the data.

4.2.3 Movie and User Effect Model

The concentration analysis revealed asymmetry on the user side. The top 10% of most active users generated 44.54% of all ratings while the bottom 30% had rated at most 36 titles each. This finding indicates the presence of systematic user level rating tendencies, whereby individual users exhibit a consistent directional rating bias a behavioral characteristics that a model accounting only for movie effect is unable to capture. To account for this limitation, a user specific effect term b_u is introduced, representing the average deviation of user μ from the global mean after the movie effect has been accounted for. The model can therefore be written as:

$$Y_{u,i} = \mu + b_i + b_u + \epsilon_{u,i}$$

```
user_effect <- edx_train_set %>%
  left_join(movie_effect, by = "movieId") %>%
  group_by(userId) %>%
  summarize(b_u = mean(rating - mu_hat - b_i), .groups = "drop")

movie_user_predict <- edx_test_set %>%
  left_join(movie_effect, by = "movieId") %>%
  left_join(user_effect, by = "userId") %>%
  mutate(pred = mu_hat + b_i + b_u) %>%
  pull(pred)
```

```

movie_user_rmse <- RMSE(edx_test_set$rating, movie_user_predict)
rmse_result <- rmse_result %>% add_row(Model = "Movie and User Effects Model",
                                       RMSE = movie_user_rmse)

cat(sprintf("Movie and user effects model RMSE: %.3f\n", movie_user_rmse))

```

Movie and user effects model RMSE: 0.866

The movie and user effects model extends the movie effect model by explicitly capturing how individual users systematically deviate from the global average, even after accounting for the movie bias. Rather than assuming uniform rating behavior across users once movie effect are removed, this model jointly estimates both movie specific and user specific tendencies. Incorporation of this additional bias correction further reduces prediction error demonstrating that the movie effect model alone is insufficient to capture the full extent of user specific rating bias present in the data.

4.2.4 Temporal Effect

The temporal analysis revealed a consistent downward trend in mean rating scores declining from 4.0 in 1995 to approximately 3.45 in 2009. This indicates that the period in which a rating was submitted exerts a systematic influence beyond the characteristics of individual users or movies. This suggests the presence of a time dependent bias in the data, a source of variation that a model capturing only movie and user effect is unable to account for. To address this limitation, a temporal bias term b_t is introduced, representing the deviation in the global rating baseline attributable to the rating period, estimated after both movie and user effects have been incorporated. The model can therefore be written as:

$$Y_{u,i,t} = \mu + b_i + b_u + b_t + \epsilon_{u,i,t}$$

```

temporal_effect <- edx_train_set %>%
  left_join(movie_effect, by = "movieId") %>%
  left_join(user_effect, by = "userId") %>%
  mutate(date = round_date(as_datetime(timestamp), unit = "week")) %>%
  group_by(date) %>%
  summarize(b_t = mean(rating - mu_hat - b_i - b_u), .groups = "drop")

temporal_model_pred <- edx_test_set %>%
  mutate(date = round_date(as_datetime(timestamp), unit = "week")) %>%
  left_join(movie_effect, by = "movieId") %>%
  left_join(user_effect, by = "userId") %>%
  left_join(temporal_effect, by = "date") %>%
  mutate(pred = mu_hat + b_i + b_u + b_t) %>%
  pull(pred)

temporal_rmse <- RMSE(edx_test_set$rating, temporal_model_pred)
rmse_result <- rmse_result %>% add_row(Model = "Temporal Effect Model",
                                       RMSE = temporal_rmse)

cat(sprintf("Temporal effect model RMSE: %.5f\n", temporal_rmse))

```

Temporal effect model RMSE: 0.86584

The temporal effect model extends the movie and user effects model by explicitly capturing systematic shifts in rating behavior over time. Instead of assuming that ratings remain stationary once movie and user biases has been removed. This model jointly estimates temporal trends at the weekly level alongside movie and user specific effects. The incorporation of this time based correction further reduces prediction error, confirming that temporal dynamics constitute a meaningful and previously unaccounted source of variation.

4.2.5 Genre Effect

The Exploratory Data Analysis revealed a substantial imbalance in genre distribution across the dataset. While the most prevalent genres account for nearly half of all item records, the least represented genres collectively constitute fewer than 5%. This disparity suggests that genre has an independent influence on ratings that persists beyond what movie specific quality, user preference or temporal trends can explain. To account for this, an additional source of system variation, a genre specific bias term b_g is introduced. The model can therefore be written as:

$$Y_{u,i,t,g} = \mu + b_i + b_u + b_t + b_g + \epsilon_{u,i,t,g}$$

```
genre_effect <- edx_train_set %>%
  mutate(date = round_date(as_datetime(timestamp), unit = "week")) %>%
  left_join(movie_effect, by = "movieId") %>%
  left_join(user_effect, by = "userId") %>%
  left_join(temporal_effect, by = "date") %>%
  separate_rows(genres, sep = "\\|") %>%
  group_by(genres) %>%
  summarize(b_g = mean(rating - mu_hat - b_i - b_u - b_t))

genre_model_pred <- edx_test_set %>%
  mutate(row_num = row_number(),
         date = round_date(as_datetime(timestamp), unit = "week")) %>%
  separate_rows(genres, sep = "\\|") %>%
  left_join(movie_effect, by = "movieId") %>%
  left_join(user_effect, by = "userId") %>%
  left_join(temporal_effect, by = "date") %>%
  left_join(genre_effect, by = "genres") %>%
  group_by(row_num) %>%
  summarize(pred = mean(mu_hat + b_i + b_u + b_t + b_g)) %>%
  arrange(row_num) %>%
  pull(pred)

genre_rmse <- RMSE(edx_test_set$rating, genre_model_pred)
rmse_result <- rmse_result %>% add_row(Model = "Genre Effect Model",
                                       RMSE = genre_rmse)

cat(sprintf("Genre effect model RMSE: %.5f\n", genre_rmse))
```

Genre effect model RMSE: 0.86573

By incorporating a genre level correction, the model identifies a distinct source of variation in ratings that earlier effect terms were unable to capture. The resulting RMSE of 0.86573 confirms that the inclusion of the genre effect yields a further reduction in prediction error. This outcome underscores the role of genre as a statistically meaningful predictive feature and demonstrates its contribution to improving overall model accuracy.

4.3 Regularization of the Baseline Model

The baseline model estimates movie, user, temporal and genre effects directly from the training data. However, these estimates can be unreliable when derived from a limited number of observations. For example, movie rated only once, will yield an extreme bias estimate that is unlikely to generalize to unseen data. Regularization addresses this issue by penalizing large effect estimates obtained from sparse evidence, shrinking them toward zero in proportion to a tuning parameter λ . The regularization effect for any group is computed as:

$$\hat{b} = \frac{\sum(r - \text{appropriate effect})}{n + \lambda}$$

4.3.1 Selecting a Tuning Parameter

Since λ is not analytically derived, it is selected by minimizing RMSE evaluated on the test set over a predefined candidate grid. In this setup, `edx_train_set` is used to estimate all effect terms for each candidate value of λ , while `edx_test_set` serves as evaluation set against which RMSE is measured.

To balance search precision with computational efficiency, a two-stage grid search strategy is used.

In the first stage, a coarse grid over $\lambda \in \{0, 0.25, 0.5, \dots, 10\}$ is evaluated to identify the approximate region containing the optimal value. In the second stage, a fine grid order $[\lambda - 1, \lambda + 1]$ with step size of 0.1 is searched around the best first-stage value of λ , yielding a more precise estimate without scanning the full parameter space at high resolution.

To avoid redundant computations across λ iterations, the timestamp-to-week format conversion and genre string disaggregation are applied to the training set prior to the search, producing two pre-computed datasets. `train_expanded`, used for estimating the temporal effect and `train_expanded_b_g`, used for the genre effect. The test set is similarly expanded once into `test_expanded`. These objects are reused across all λ evaluations in both grid stages.

The evaluation logic for each λ is encapsulated in a dedicated function `compute_rmse`, which sequentially estimates all four effects terms, generates predictions on the test set by summing the global mean with the movie, user, temporal and genre effects, averaging within each row to handle duplicate joins and then returns the RMSE against the true observed ratings.

```
train_expanded <- edx_train_set %>%
  mutate(date = round_date(as_datetime(timestamp), unit = "week"))

train_expanded_b_g <- train_expanded %>%
  separate_rows(genres, sep = "\\|")

test_expanded <- edx_test_set %>%
  mutate(row_num = row_number(),
         date = round_date(as_datetime(timestamp), unit = "week")) %>%
  separate_rows(genres, sep = "\\|")

compute_rmse <- function(lambda) {
  movie_effect <- edx_train_set %>%
    group_by(movieId) %>%
    summarize(b_i = sum(rating - mu_hat) / (n() + lambda))

  user_effect <- edx_train_set %>%
    left_join(movie_effect, by = "movieId") %>%
    group_by(userId) %>%
    summarize(b_u = sum(rating - mu_hat - b_i) / (n() + lambda))

  temporal_effect <- train_expanded %>%
    left_join(movie_effect, by = "movieId") %>%
    left_join(user_effect, by = "userId") %>%
    group_by(date) %>%
    summarize(b_t = sum(rating - mu_hat - b_u - b_i) / (n() + lambda))

  genre_effect <- train_expanded_b_g %>%
    left_join(movie_effect, by = "movieId") %>%
```

```

    left_join(user_effect, by = "userId") %>%
    left_join(temporal_effect, by = "date") %>%
    group_by(genres) %>%
    summarize(b_g = sum(rating - mu_hat - b_i - b_u - b_t) / (n() + lambda))

pred <- test_expanded %>%
  left_join(movie_effect, by = "movieId") %>%
  left_join(user_effect, by = "userId") %>%
  left_join(temporal_effect, by = "date") %>%
  left_join(genre_effect, by = "genres") %>%
  group_by(row_num) %>%
  summarize(pred = mean(mu_hat + b_i + b_u + b_t + b_g)) %>%
  arrange(row_num) %>%
  pull(pred)

RMSE(edx_test_set$rating, pred)
}

lambdas_s1 <- seq(0, 10, 0.25)
rmses_s1 <- sapply(lambdas_s1, compute_rmse)
best_lambda <- lambdas_s1[which.min(rmses_s1)]
cat(sprintf("Stage 1 best lambda: %.2f\n", best_lambda))

```

Stage 1 best lambda: 5.00

```

rmse_result <- rmse_result %>%
  add_row(Model = "Regularized Genre Effect Model (first stage grid)",
          RMSE = min(rmses_s1))

lambda_s2 <- seq(best_lambda - 1, best_lambda + 1, 0.1)
rmses_s2 <- sapply(lambda_s2, compute_rmse)
best_lambda <- lambda_s2[which.min(rmses_s2)]
cat(sprintf("Stage 2 best lambda: %.2f\n", best_lambda))

```

Stage 2 best lambda: 5.10

```

rmse_result <- rmse_result %>%
  add_row(Model = "Regularized Genre Effect Model (second stage grid)",
          RMSE = min(rmses_s2))

```

The coarse grid search over $\lambda \in \{0, 0.25, 0.5, \dots, 10\}$ identified an initial optimum. A subsequent fine grained search over $[\lambda - 1, \lambda + 1]$ with a step size of 0.1 yielding a best value of $\lambda = 5.1$. Both stages produced an identical RMSE of 0.86501, indicating that the coarse grid had already located the true minimum with sufficient precision and that the fine search served to confirm rather than improving upon it. This outcome validates the two-stage design, specifically that the second stage provides confidence that no superior value exists within the refine search interval. The selected penalty $\lambda = 5.1$ reflects a moderated level of shrinkage pulling effects estimates towards zero for items, users, weeks and genres with limited observational support.

4.4 Matrix Factorization Model

The Matrix Factorization model is constructed directly upon the regularized bias model developed in the preceding section and is designed to capture the structured patterns that persist in the residuals after all additive bias effects have been accounted for.

As an initial step, all four bias effects are re-estimated using **best_lambda** value which is the regularization parameter selected during the grid search in the previous section.

```

movie_effect_lmb <- edx_train_set %>%
  group_by(movieId) %>%
  summarize(b_i = sum(rating - mu_hat) / (n() + best_lambda))

user_effect_lmb <- edx_train_set %>%
  left_join(movie_effect_lmb, by = "movieId") %>%
  group_by(userId) %>%
  summarize(b_u = sum(rating - mu_hat - b_i) / (n() + best_lambda))

temporal_effect_lmb <- edx_train_set %>%
  mutate(date = round_date(as_datetime(timestamp), unit = "week")) %>%
  left_join(movie_effect_lmb, by = "movieId") %>%
  left_join(user_effect_lmb, by = "userId") %>%
  group_by(date) %>%
  summarize(b_t = sum(rating - mu_hat - b_i - b_u) / (n() + best_lambda))

genre_effect_lmb <- edx_train_set %>%
  mutate(date = round_date(as_datetime(timestamp), unit = "week")) %>%
  left_join(movie_effect_lmb, by = "movieId") %>%
  left_join(user_effect_lmb, by = "userId") %>%
  left_join(temporal_effect_lmb, by = "date") %>%
  separate_rows(genres, sep = "\\|") %>%
  group_by(genres) %>%
  summarize(b_g = sum(rating - mu_hat - b_i - b_u - b_t) / (n() + best_lambda))

```

The residuals are defined as the difference between the observed rating and the regularized baseline prediction. For titles assigned to multiple genres, the genre effect is computed as the average of the individual genre specific effects across all genres associated with that movie.

```

residuals <- edx_train_set %>%
  mutate(row_num = row_number(),
         date = round_date(as_datetime(timestamp), unit = "week")) %>%
  separate_rows(genres, sep = "\\|") %>%
  left_join(movie_effect_lmb, by = "movieId") %>%
  left_join(user_effect_lmb, by = "userId") %>%
  left_join(temporal_effect_lmb, by = "date") %>%
  left_join(genre_effect_lmb, by = "genres") %>%
  group_by(row_num) %>%
  summarize(movieId = first(movieId),
            userId = first(userId),
            resid = first(rating) - mean(mu_hat + b_i + b_u + b_t + b_g))

```

These residuals serves as the target variable for the Matrix Factorization model. The **recosystem** package is used to initialize and train the recommender system. Hyperparameter tuning is performed via **r\$tune()**, conducting a search over a range of latent factor dimensions while keeping the regularization cost, learning rate, number of iterations, and number of threads fixed. The best configuration identified during tuning is subsequently used to train the final model via **r\$train()**.

```

r <- Reco()

train_reco <- data_memory(residuals$userId, residuals$movieId,
                          residuals$resid, index1 = TRUE)

set.seed(1)
opts_tune_2 <- r$tune(train_reco, opts = list(

```

```

dim = c(30, 35, 40, 45, 50),
costp_l1 = 0,
costp_l2 = 0.01,
costq_l1 = 0,
costq_l2 = 0.1,
lrate = 0.05,
niter = 30,
nthread = 1,
seed = 1,
verbose = FALSE
))

set.seed(1)
r$train(train_reco,
        opts = c(opts_tune_2$min,
                  list(niter = 30,
                       nthread = 1,
                       seed = 1,
                       verbose = TRUE)))

```

iter	tr_rmse	obj
0	0.8635	5.6107e+06
1	0.8491	5.2730e+06
2	0.8353	5.1359e+06
3	0.8268	5.0597e+06
4	0.8180	4.9908e+06
5	0.8086	4.9179e+06
6	0.7994	4.8478e+06
7	0.7909	4.7845e+06
8	0.7833	4.7285e+06
9	0.7764	4.6784e+06
10	0.7701	4.6341e+06
11	0.7644	4.5944e+06
12	0.7591	4.5590e+06
13	0.7541	4.5255e+06
14	0.7494	4.4948e+06
15	0.7451	4.4667e+06
16	0.7410	4.4403e+06
17	0.7372	4.4161e+06
18	0.7336	4.3934e+06
19	0.7301	4.3718e+06
20	0.7268	4.3519e+06
21	0.7237	4.3330e+06
22	0.7207	4.3148e+06
23	0.7179	4.2977e+06
24	0.7151	4.2823e+06
25	0.7125	4.2671e+06
26	0.7100	4.2523e+06
27	0.7076	4.2385e+06
28	0.7053	4.2256e+06
29	0.7031	4.2128e+06

The final prediction is formed as the sum of two components, specifically the bias prediction derived from the regularized effects and the matrix factorization prediction produced by `r$predict()`. This combined

approach assigns the bias terms to handle the main systematic patterns while the matrix factorization to model the remaining user-item interactions.

```

bias_pred <- edx_test_set %>%
  mutate(row_num = row_number(),
         date = round_date(as_datetime(timestamp), unit = "week")) %>%
  separate_rows(genres, sep = "\\|") %>%
  left_join(movie_effect_lmb, by = "movieId") %>%
  left_join(user_effect_lmb, by = "userId") %>%
  left_join(temporal_effect_lmb, by = "date") %>%
  left_join(genre_effect_lmb, by = "genres") %>%
  group_by(row_num) %>%
  summarize(pred = first(mu_hat) + first(b_i) + first(b_u) + first(b_t) + mean(b_g)) %>%
  arrange(row_num) %>%
  pull(pred)

test_reco <- data_memory(edx_test_set$userId,
                        edx_test_set$movieId,
                        index1 = TRUE)
mf_pred <- r$predict(test_reco, out_memory())

final_pred <- bias_pred + mf_pred

mf_rmse <- RMSE(edx_test_set$rating, final_pred)

rmse_result <- rmse_result %>%
  add_row(Model = "Matrix Factorization Model", RMSE = mf_rmse)

cat(sprintf("Matrix Factorization RMSE: %.5f\n", mf_rmse))

```

Matrix Factorization RMSE: 0.79463

The combined bias and matrix factorization model achieves an RMSE of 0.79463 on the `edx_test_set`, the lowest value across all models evaluated in this study. This result confirms that capturing the residual user-item interactions yields a meaningful improvement over the regularized baseline.

The regularized baseline model achieved an RMSE of 0.86501, while the MF model reduces an RMSE to 0.79463 with an absolute reduction of 0.07038, corresponding to a relative improvement of 8.14%.

5 Holdout Set Evaluation

The `final_holdout_test` set was withheld entirely throughout all stages of model development and hyperparameter tuning. It is used exclusively at this stage to report the generalization performance of the best-performing model selected during model development phase.

The best-performing model identified during development is the **Matrix Factorization model**. This model integrates regularized bias effects with a matrix factorization component trained on the residuals using the `recosystem` library. All model parameters and bias terms estimates were derived exclusively from the `edx_train_set` and are applied here without any retraining or parameter adjustment.

```

holdout_bias_pred <- final_holdout_test %>%
  mutate(row_num = row_number(),
         date = round_date(as_datetime(timestamp), unit = "week")) %>%
  separate_rows(genres, sep = "\\|") %>%
  left_join(movie_effect_lmb, by = "movieId") %>%
  left_join(user_effect_lmb, by = "userId") %>%

```

```

left_join(temporal_effect_lmb, by = "date") %>%
left_join(genre_effect_lmb, by = "genres") %>%
group_by(row_num) %>%
summarize(pred = first(mu_hat) + first(b_i) + first(b_u) + first(b_t) + mean(b_g)) %>%
arrange(row_num) %>%
pull(pred)

holdout_reco <- data_memory(final_holdout_test$userId,
                           final_holdout_test$movieId,
                           index1 = TRUE)
holdout_mf_pred <- r$predict(holdout_reco, out_memory())

final_holdout_pred <- holdout_bias_pred + holdout_mf_pred

holdout_mf_rmse <- RMSE(final_holdout_test$rating, final_holdout_pred)

rmse_result <- rmse_result %>%
  add_row(Model = "Matrix Factorization Model (final_holdout_set)", RMSE = holdout_mf_rmse)

cat(sprintf("Matrix Factorization Model (final_holdout_set): %.5f\n", holdout_mf_rmse))

```

Matrix Factorization Model (final_holdout_set): NA

6 Conclusion

This project explored a progression of recommendation system models of increasing complexity. The model pipeline began with a global mean baseline and was progressively extended through the incorporation of bias terms that captured movie specific, user specific, date and genre level effects. Following the establishment of this multi-component additive structure, regularization was applied to mitigate overfitting arising from sparse observations and a matrix factorization component was subsequently introduced to model the residual user-item interactions that the bias terms alone were unable to account for. The best-performing model combined regularized bias effects with a matrix factorization component trained on the residuals, yielding strong predictive performance across all evaluated configurations.

The Matrix Factorization model evaluated on the `final-holdout-test` set achieved RMSE of: **0.79426**

This result confirms that the model generalizes effectively beyond the training data and meets the target performance threshold. The application of regularization across all bias terms proved essential in preventing overfitting to sparsely observed users and movies, while matrix factorization component provided a valuable improvement over the regularized approach, demonstrating the value of capturing latent structure in the residuals.